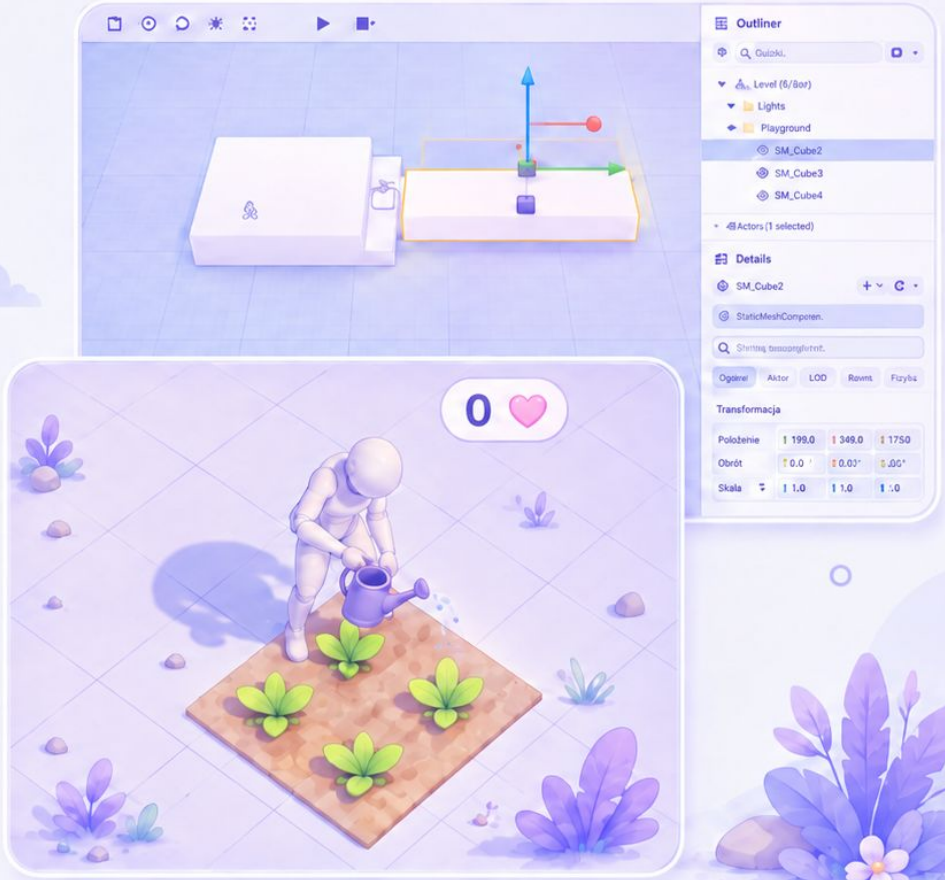
UNREAL ENGINE

Matematyka w grach

WYKŁAD



Czym jest punkt?

Punkt to podstawowe pojęcie w geometrii.

Definicja punktu

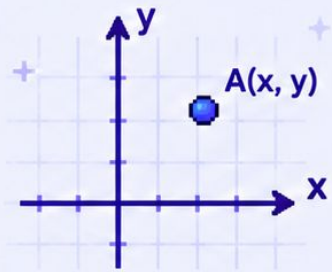
Punkt to obiekt geometryczny, który nie ma wymiarów — nie ma długości, szerokości ani wysokości.



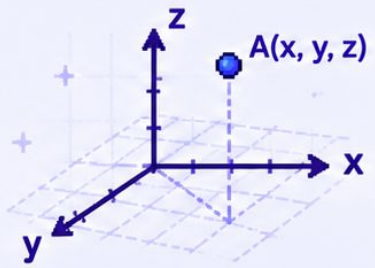
Układ współrzędnych

Położenie punktu opisujemy za pomocą współrzędnych.

W 2D: (x, y)



W 3D: (x, y, z)



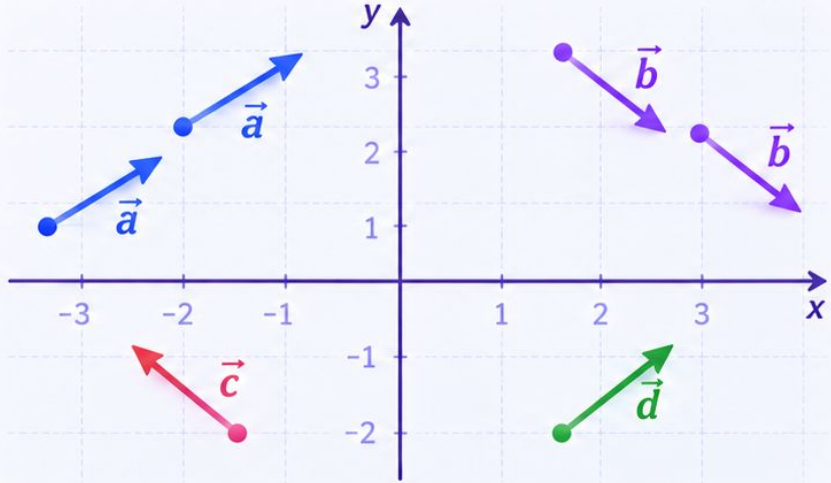
W 2D używamy 2 współrzędnych, a w 3D — 3 współrzędnych.

Wektor

Wielkość mająca kierunek i wartość

Wektor to wielkość opisana przez **długość** i **kierunek**.

- Przedstawiamy go jako strzałkę.
- Zwrot pokazuje, w którą stronę działa.



Wektor nie ma miejsca przyłożenia. | Można go przesuwać równolegle bez zmiany jego długości i kierunku.

Jeden Vector, dwa znaczenia

W Unreal Engine typ Vector przechowuje X, Y, Z — zarówno dla punktów, jak i wektorów.

Punkt (pozycja)

Gdzie obiekt ma się znaleźć
w przestrzeni 3D?



`Vector(300, 120, 80)` → `SetWorldLocation`

Wektor (przesunięcie)

O ile i w którą stronę
ma przesunąć się obiekt?



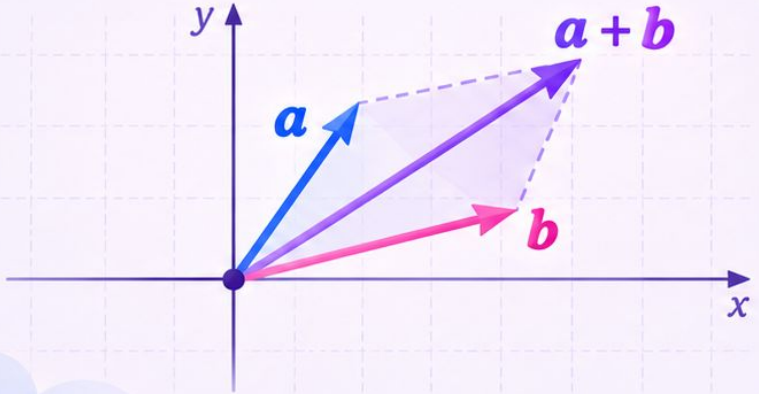
`Vector(300, 120, 80)` → `AddWorldOffset`

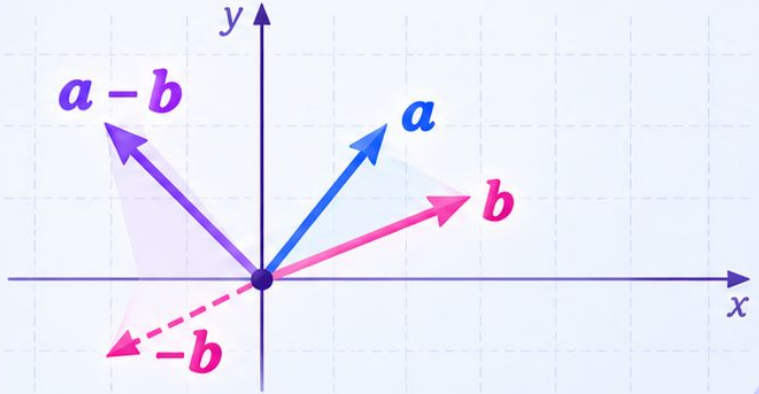
Ten sam typ danych — znaczenie zależy od kontekstu użycia.

Dodawanie i odejmowanie wektorów

$$\vec{a} + \vec{b} = (a_x + b_x,\ a_y + b_y)$$

$$\vec{a} - \vec{b} = \vec{a} + (-\vec{b}) = (a_x - b_x,\ a_y - b_y)$$





Zadanie

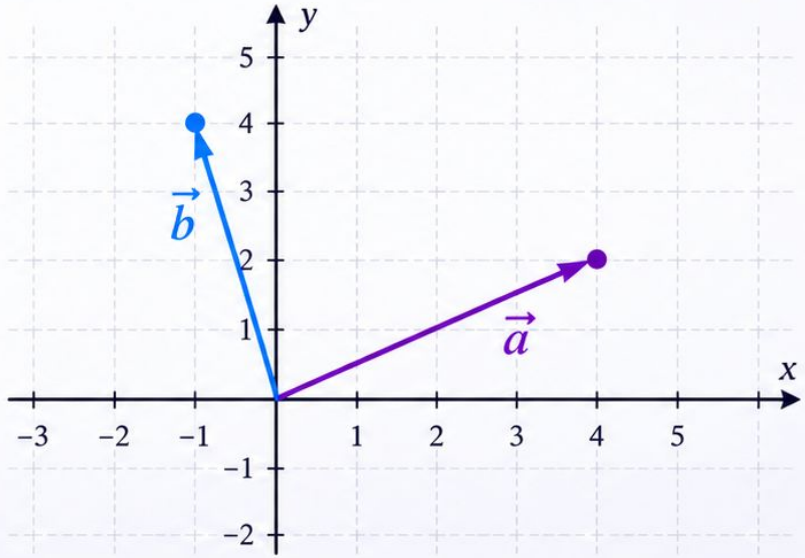
Dodawanie i odejmowanie wektorów

Treść zadania

Dane są wektory

$$\vec{a} = (3, 2) \quad \text{oraz} \quad \vec{b} = (-1, 4).$$

1. Oblicz $\vec{a} + \vec{b}$.
2. Oblicz $\vec{a} - \vec{b}$.
3. Zaznacz wektory $\vec{a}$ i $\vec{b}$ na układzie współrzędnych.
4. Zaznacz również wektor $\vec{a} + \vec{b}$.



Wskazówka

Aby dodać (odjąć) wektory, dodaj (odejmij) ich odpowiadające sobie współrzędne.

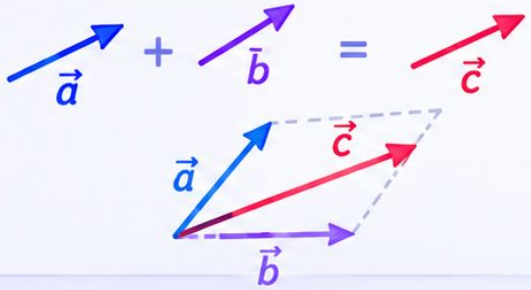
$$\vec{a} + \vec{b} = (a_1 + b_1,\ a_2 + b_2)$$

$$\vec{a} - \vec{b} = (a_1 - b_1,\ a_2 - b_2)$$

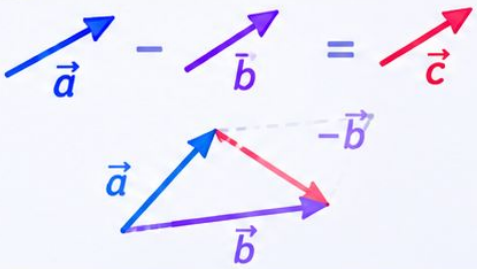
Działania na punktach i wektorach

Co otrzymamy w wyniku? – krótkie podsumowanie

1 Wektor + wektor = wektor

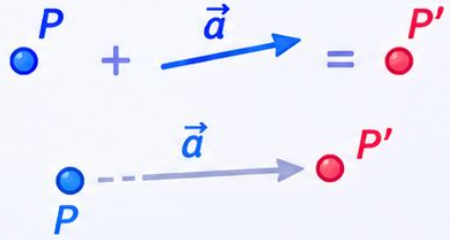


2 Wektor – wektor = wektor



3 Punkt + wektor = punkt

(przesunięcie punktu)

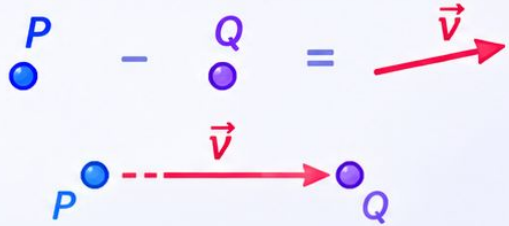


4 Punkt + punkt = bezużyteczne śmieci



Dodawanie dwóch punktów nie ma sensu w geometrii – nie daje żadnego użytecznego wyniku.

5 Punkt – punkt = wektor



Wypadkowa sił

Czym jest i jak wykorzystać ją w Unreal Engine

Czym jest wypadkowa?

Wypadkowa sił to jedna siła, która **zastępuje kilka sił** działających na obiekt i daje **taki sam efekt ruchu**, jak wszystkie te siły razem.

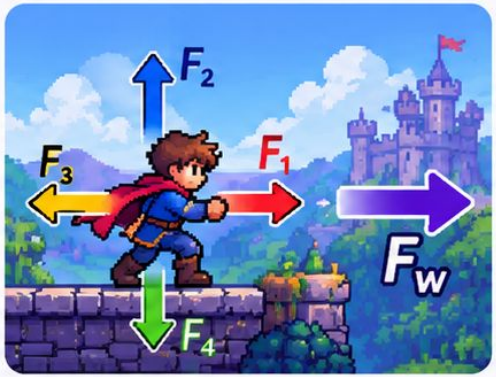
Jak obliczyć wypadkową?

W wersji 2D dodajemy siły wektorowo (składowe w osi X i Y):

$$\vec{F_w} = \vec{F_1} + \vec{F_2} + \vec{F_3} + \cdots$$

Wynik to pojedynczy wektor – wypadkowa, która ma określony kierunek i wartość.

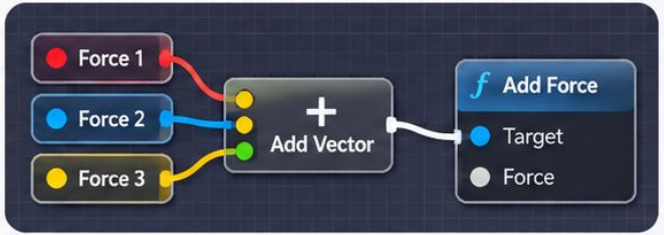
Przykład wizualny



Wypadkowa $\vec{F_w}$ to pojedyncza siła, która powoduje taki sam ruch jak wszystkie siły $\vec{F_1}, \vec{F_2}, \vec{F_3}, \vec{F_4}$ **razem.**

Jak wykorzystać wypadkową w Unreal Engine?

W Unreal Engine wypadkową siłę stosujemy do obiektu, aby uzyskać jego ruch zgodny ze wszystkimi działającymi na niego siłami.



1. **Oblicz wypadkową** – zsumuj wektory sił (np. node „Add Vector").
2. Użyj wypadkowej w funkcji **Add Force** (np. na komponencie Static Mesh lub Character Movement).
3. Obiekt otrzyma jedną siłę, która daje ten sam efekt co wszystkie siły **naraz**.

Wypadkowa pozwala uprościć obliczenia i szybko przewidzieć, w którą stronę i jak mocno poruszy się obiekt.

Punkt − punkt = wektor

Różnica współrzędnych dwóch punktów tworzy **wektor między nimi**, co pozwala określić odległość i kierunek między punktami.

Wektor między punktami

Wektor od A do B: **punkt końcowy** minus **punkt początkowy**. Dla punktów $A(x_A, y_A)$ i $B(x_B, y_B)$ mamy:

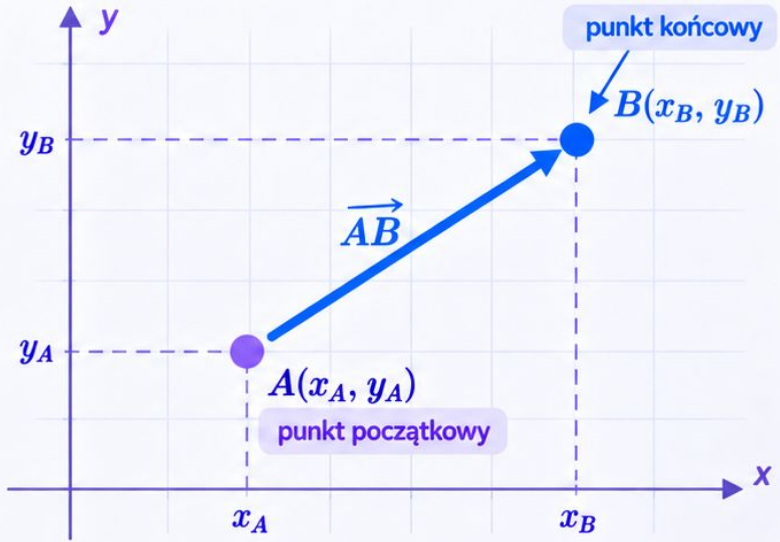
$$\overrightarrow{AB} = (x_B - x_A,\ y_B - y_A)$$

Najpierw współrzędne punktu, do którego wskazuje grot wektora (**punkt końcowy B**), a następnie współrzędne **punktu początkowego** (A).

Odległość między punktami

Długość tego wektora jest równa odległości między punktami A i B:

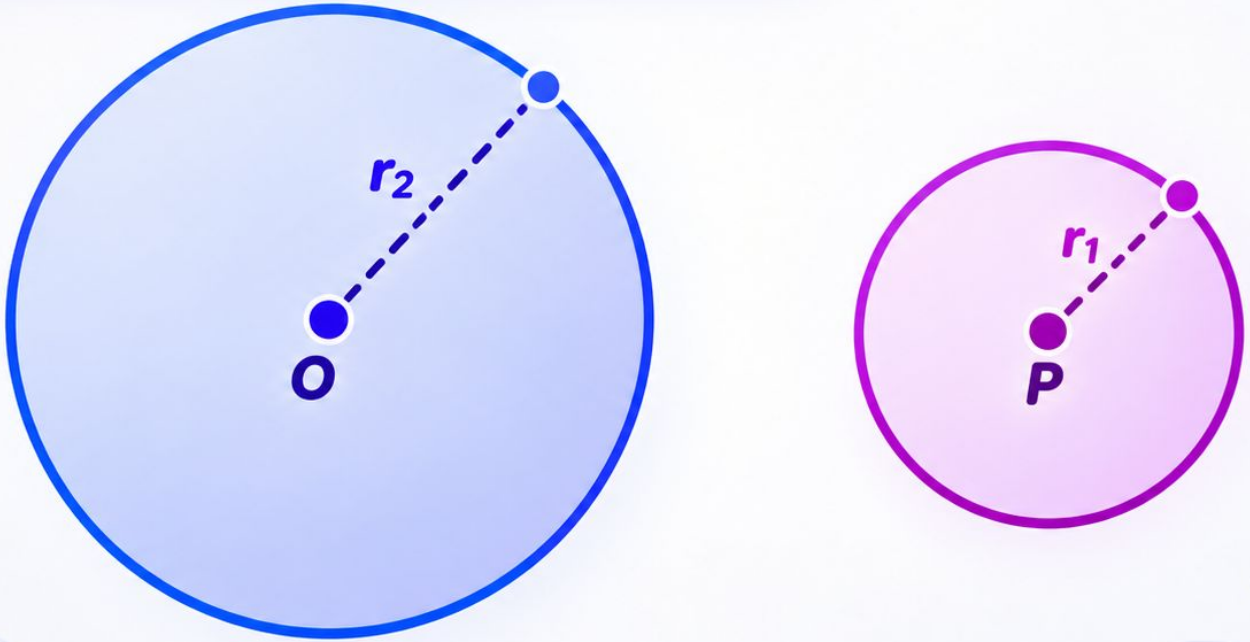
$$|\overrightarrow{AB}| = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$



Wniosek

Punkt − punkt daje nam **wektor** między punktami, a długość tego wektora określa **odległość** między nimi.

Czy zachodzi kolizja?



Jaka jest odległość od przeciwnika?

Zadanie • oblicz odległość na mapie gry

ETAP 1/2

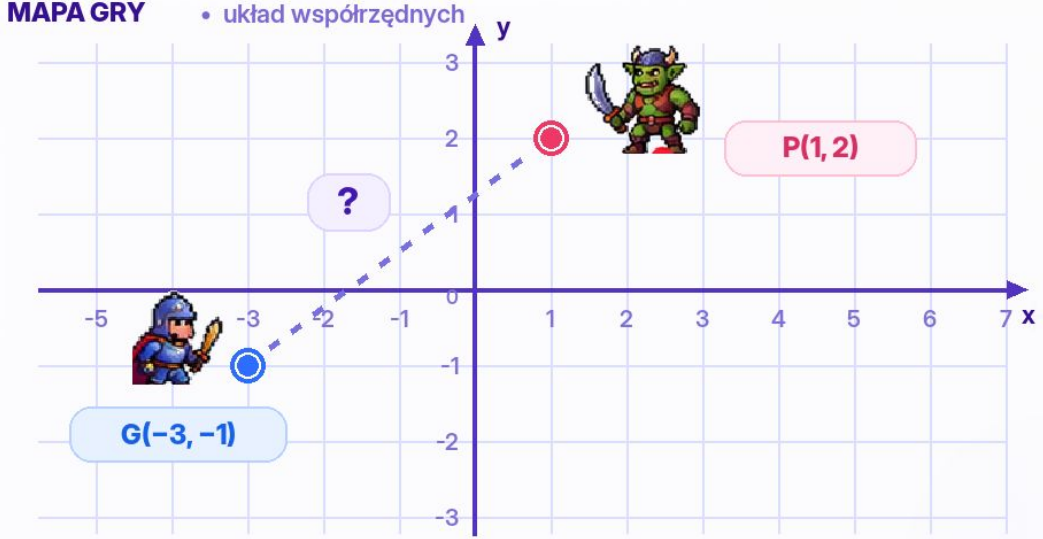
Twoje zadanie

Na mapie gry zaznaczono dwa punkty:

- **Gracz: G(−3, −1)**
- **Przeciwnik: P(1, 2)**

Oblicz odległość między nimi
w linii prostej.

MAPA GRY • układ współrzędnych



1 kratka = 1 jednostka odległości

Jaka jest odległość od przeciwnika?

Zadanie i rozwiązanie • krok po kroku

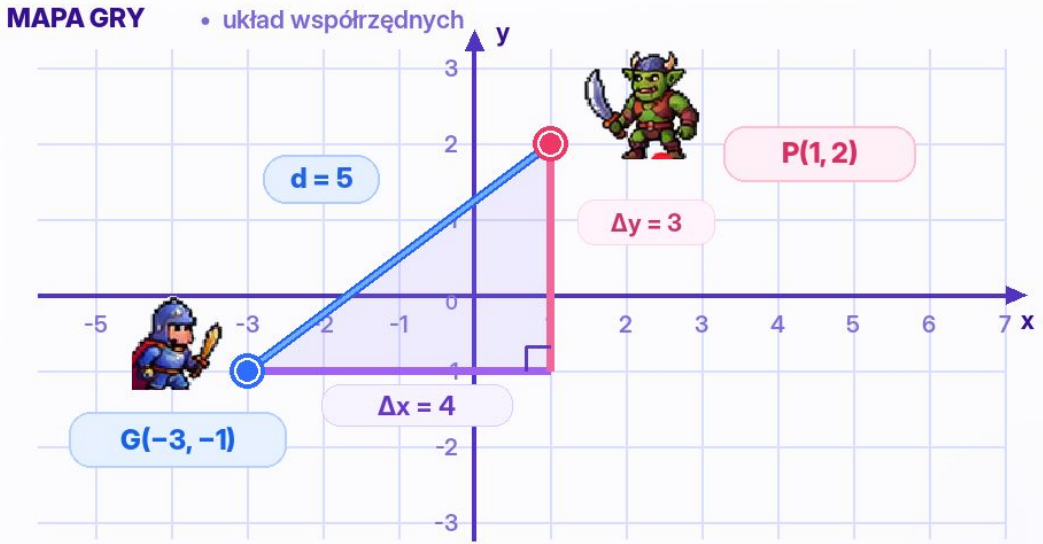
ETAP 2/2

Rozwiązanie

Zadanie: G(−3, −1) i P(1, 2)

1. Różnica w poziomie:
   $\Delta x = 1 - (-3) = 4$
2. Różnica w pionie:
   $\Delta y = 2 - (-1) = 3$
3. Twierdzenie Pitagorasa:
   $d = \sqrt{(4^2 + 3^2)}$

   $d = \sqrt{25} = 5$



Odpowiedź: Odległość wynosi 5 jednostek.

Czy przeciwnik jest w zasięgu N?

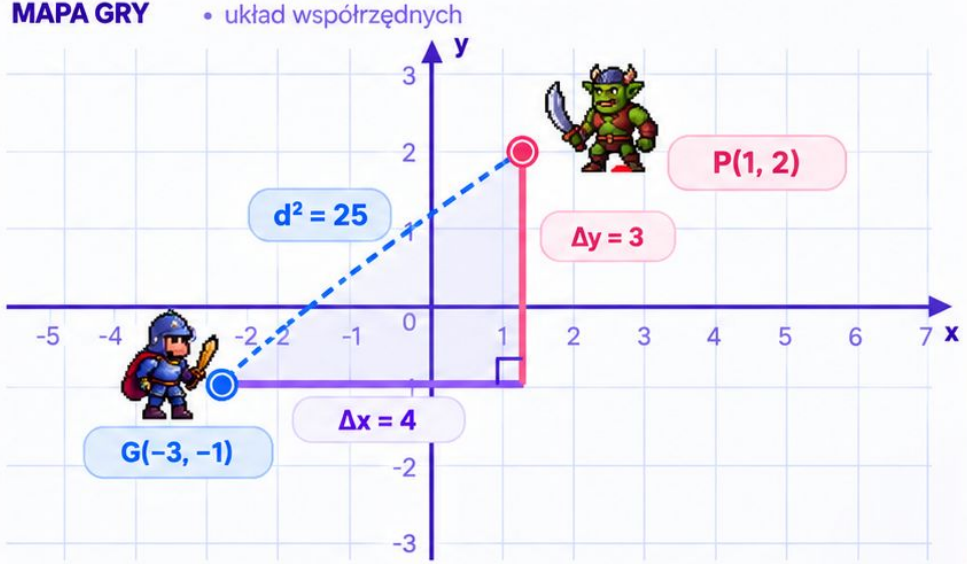
Zamiast liczyć pierwiastek – porównujemy wartości w kwadracie

Dla komputera obliczanie pierwiastka jest trudniejsze niż dodawanie i mnożenie. Dlatego zamiast sprawdzać, czy $d \le N$, porównujemy $d^2 \le N^2$.

Jeśli $(x_2 - x_1)^2 + (y_2 - y_1)^2 \le N^2$, to przeciwnik jest w zasięgu.

Przykład dla naszych punktów:

1. Gracz: G(−3, −1), Przeciwnik: P(1, 2)
2. $\Delta x = 1 - (-3) = 4$
3. $\Delta y = 2 - (-1) = 3$
4. $d^2 = 4^2 + 3^2 = 16 + 9 = 25$



Dla N = 4: $N^2 = 16$, a $25 > 16$, więc przeciwnik **NIE jest w zasięgu**.

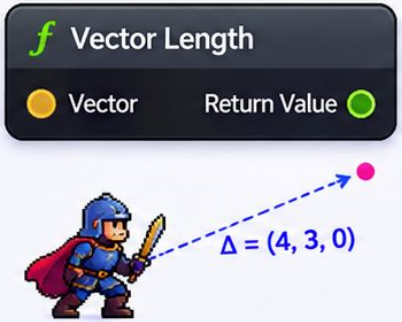
Dla N = 5: $N^2 = 25$, a $25 \le 25$, więc przeciwnik **JEST w zasięgu**.

W programowaniu często porównujemy odległości w kwadracie, bo to **szybsze i prostsze dla komputera.**

Vector Length czy Vector Length Squared?

Unreal Engine Blueprint — obliczanie odległości i sprawdzanie zasięgu

Vector Length



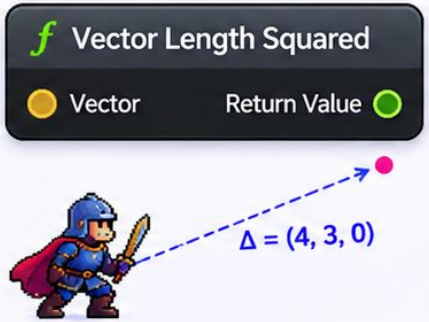
Oblicza długość wektora (odległość od punktu 0,0,0).

$$d = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$$

Przykład:

$\Delta = (4, 3, 0) \rightarrow d = 5$

Vector Length Squared



Zwraca kwadrat długości wektora — **bez pierwiastka.**

$$d^2 = \Delta x^2 + \Delta y^2 + \Delta z^2$$

Przykład:

$\Delta = (4, 3, 0) \rightarrow d^2 = 25$

Czy przeciwnik jest w zasięgu N?

Zamiast obliczać odległość i porównywać z N, porównujemy kwadrat odległości z kwadratem zasięgu.

Delta = PozycjaPrzeciwnika – PozycjaGracza → Vector Length Squared (Vector → Return Value) → $d^2 \leq N^2$



Do sprawdzania zasięgu używaj Vector Length Squared — unikniesz obliczania pierwiastka.

Pierwiastek jest zwykle droższy obliczeniowo niż dodawanie i mnożenie.

Wektor znormalizowany

Ten sam kierunek, ale długość równa 1

Czym jest?

Wektor znormalizowany to wektor o długości 1, który zachowuje ten sam kierunek co wektor pierwotny. Używa się go do określania kierunków, niezależnie od odległości.

Wzór normalizacji

$$\vec{v}_{norm} = \frac{\vec{v}}{||\vec{v}||}$$

gdzie:

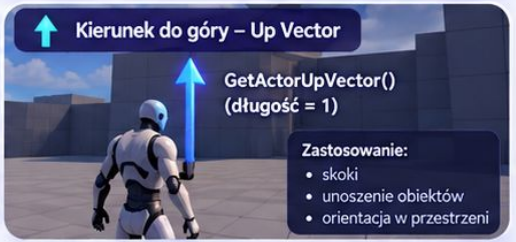
- $\vec{v}$ – wektor pierwotny
- $||\vec{v}||$ – długość (moduł) wektora
- $\vec{v}_{norm}$ – wektor znormalizowany

Przykłady z Unreal Engine

GetActorForwardVector()	Kierunek, w który patrzy aktor (np. strzelanie, ruch do przodu).
GetActorRightVector()	Kierunek w prawo względem aktora (np. ruch na boki, unikanie).
GetActorUpVector()	Kierunek do góry (np. skoki, unoszenie).
GetSafeNormal()	Normalizuje dowolny wektor (naprz. kierunek do celu).









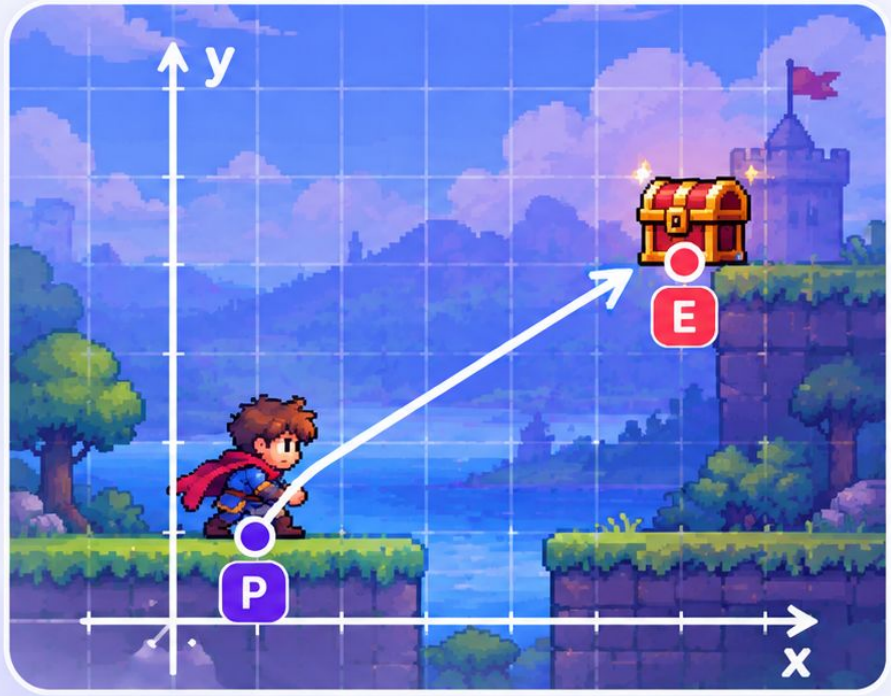


Wektor znormalizowany i wektor normalny to nie to samo!

Wektor znormalizowany ma długość 1 i zachowuje kierunek. Wektor normalny to wektor prostopadły do powierzchni (używany np. w oświetleniu i kolizjach).

Jak przesunąć gracza bliżej

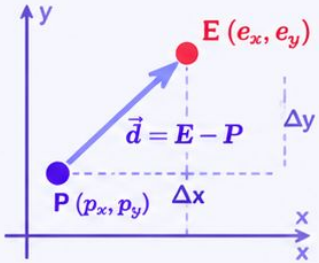
Przesuwamy gracza w kierunku celu (E) o ustaloną odległość.



1 Oblicz kierunek do celu

Tworzymy wektor z gracza (P) do celu (E).

$$\vec{d} = E - P$$



2 Przesuń gracza o ustaloną odległość

Normalizujemy wektor kierunku i mnożymy go przez wybraną odległość.

$$P_{nowy} = P + \text{znormalizuj}(E - P) \cdot d$$

- **P** pozycja gracza (p_x, p_y)
- **E** pozycja celu (e_x, e_y)
- **d** odległość przesunięcia (np. prędkość × czas)

Mnożenie wektorów

Dla wektorów mamy kilka typów mnożenia, które służą do różnych celów w Unreal Engine.

1 Mnożenie przez skalar
(Vector * Float)

Zmienia długość wektora;
dla dodatniego skalaru zachowuje kierunek.

$$\boldsymbol{v} \cdot s = (x \cdot s, y \cdot s, z \cdot s)$$



W UNREAL ENGINE:
Skalowanie długości wektora
(np. prędkość, siła, odległość).

2 Iloczyn skalarny
(Dot Product)

Zwraca jedną liczbę określającą,
jak bardzo wektory są do siebie podobne.

$$\boldsymbol{a} \cdot \boldsymbol{b} = |a||b| \cos\theta$$



W UNREAL ENGINE:
Sprawdzanie kierunku i kąta
(np. czy obiekt jest przed nami,
wykrywanie pola widzenia).

3 Iloczyn wektorowy
(Cross Product)

Zwraca wektor prostopadły do obu
wektorów wejściowych.

$$\boldsymbol{a} \times \boldsymbol{b}$$



W UNREAL ENGINE:
Wyznaczanie prostopadłego wektora
(np. normalna powierzchni,
wektor boczny, obliczenia orientacji).

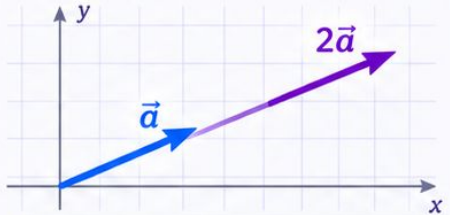
Iloczyn wektora przez skalar

Jak liczba zmienia długość i zwrot wektora

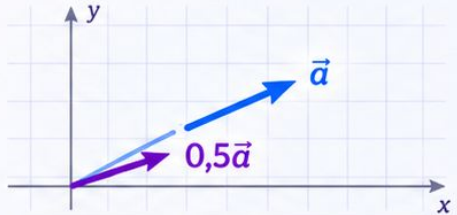
Najważniejsze zasady

1. $k > 1$ – wektor wydłuża się.
2. $0 < k < 1$ – wektor się skraca.
3. $k < 0$ – wektor zmienia zwrot.
4. $k = 0$ – otrzymujemy wektor zerowy.
5. $|k \cdot \boldsymbol{a}| = |k| \cdot |\boldsymbol{a}|$

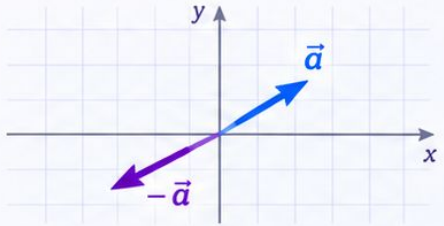
Mnożenie wektora przez skalar nie zmienia kierunku prostej, a jedynie długość i ewentualnie zwrot wektora.



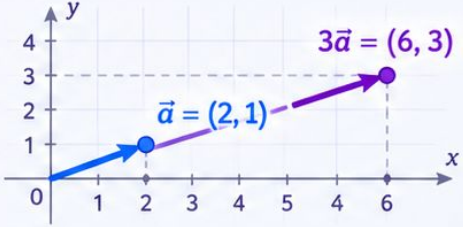
k = 2



k = 0,5



k = −1



Przykład współrzędnych

Iloczyn wektora przez skalar zmienia jego długość, a dla liczby ujemnej także zwrot.

1 Iloczyn skalarny

Czym jest i co właściwie opisuje?

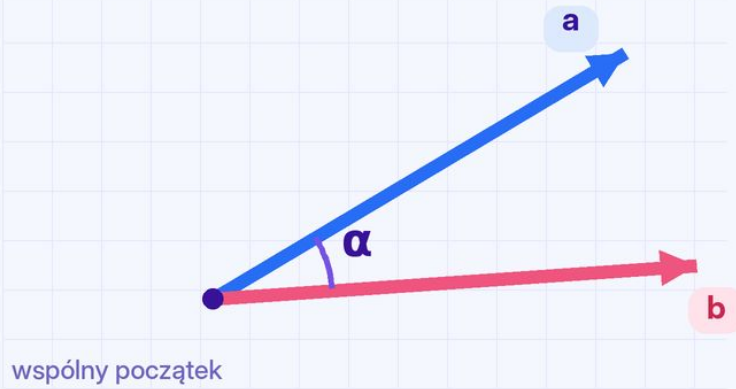
DEFINICJA

To działanie na dwóch wektorach, którego wynikiem jest liczba.

$$a \cdot b$$

Ta liczba zależy od długości wektorów i kąta między nimi.

Dwa wektory, jeden kąt



Iloczyn skalarny = liczba, nie wektor.

2 Dwa sposoby obliczania

Dwa wzory matematyczne i node z Unreal Engine Blueprint.

1 Ze współrzędnych

Dla wektorów w 2D:

$$a \cdot b = a_1 b_1 + a_2 b_2$$

Mnożysz współrzędne parami i dodajesz wyniki.

2 Z długości i kąta

Gdy znasz kąt α między wektorami:

$$a \cdot b = |a| \cdot |b| \cdot \cos \alpha$$

Mnożysz długości wektorów i cosinus kąta α.

UNREAL ENGINE • BLUEPRINT

Node: Dot Product (Vector)

Dwa wejścia: Vector → Wynik: Float



Unreal Engine: Dot Product oblicza iloczyn skalarny.

3 Przykład obliczenia

Policzmy iloczyn skalarny i narysujmy oba wektory.

DANE

$\mathbf{a} = (2, 3)$

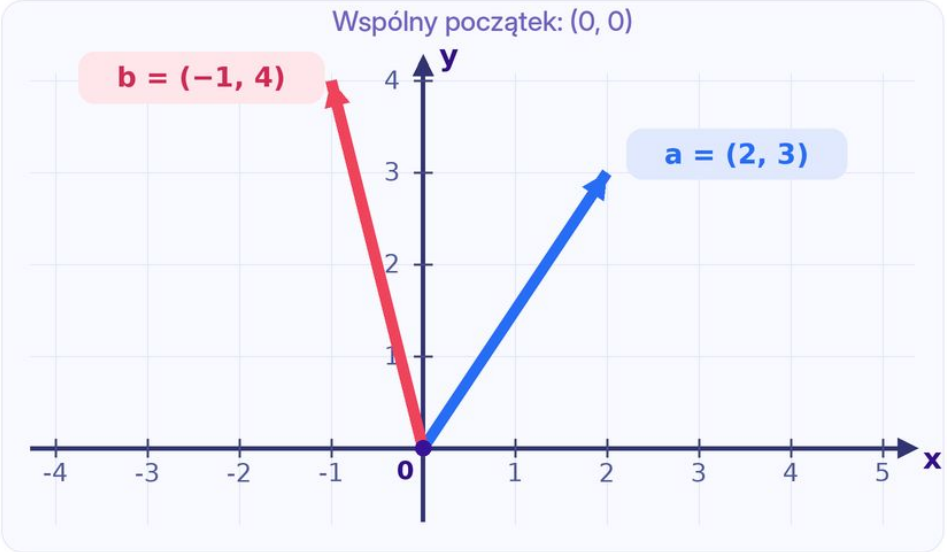
$\mathbf{b} = (-1, 4)$

PODSTAW DO WZORU

$\mathbf{a} \cdot \mathbf{b} = 2 \cdot (-1) + 3 \cdot 4$

$= -2 + 12$

$\mathbf{a} \cdot \mathbf{b} = 10$



W tym przykładzie wynik jest dodatni: 10.

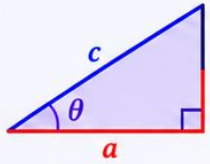
Czym jest cosinus?

Cosinus kąta – definicja i intuicja

Definicja

Cosinus kąta to wartość, która mówi, jak bardzo dwa kierunki są do siebie „zgodne".
Dla kąta θ ($0° – 180°$) cosinus przyjmuje wartości od 1 do –1.

$$\cos\theta = \frac{\text{przyprostokątna przyległa}}{\text{przeciwprostokątna}} = \frac{a}{c}$$



- c – przeciwprostokątna
- a – przyprostokątna przyległa do kąta θ
- θ – mierzony kąt

Intuicja – co mówi wartość cosinusa?

Cosinus określa, jak bardzo kierunek jednego wektora pokrywa się z kierunkiem drugiego.

$\cos 0° = 1$

Kierunki są **identyczne** (w pełni zgodne).



$\cos 90° = 0$

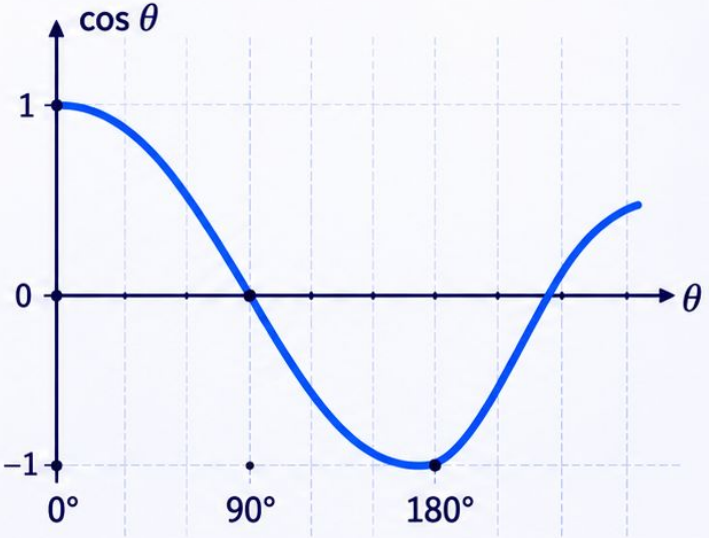
Kierunki są **prostopadłe** (nie są zgodne).

$\cos 180° = -1$

Kierunki są przeciwnie skierowane (w pełni przeciwne).

Wykres funkcji cosinus

Funkcja cosinus zmienia się w sposób falowy w zależności od kąta θ.



Dla naszych zastosowań (wykrywanie przeciwnika) interesuje nas zwykle zakres **0° – 180°**, ponieważ cosinus kąta między wektorami zawiera się w przedziale **[–1, 1]**.

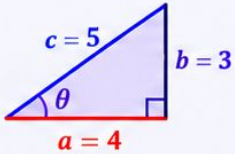
Przykładowe wartości

Najczęściej spotykane kąty i odpowiadające im wartości cosinusa.

Kąt θ	$\cos\theta$	Opis
0°	1	Kierunki identyczne
30°	≈ 0.866	Bardzo podobne kierunki
45°	≈ 0.707	Umiarkowanie podobne
60°	0.5	Mniej podobne
90°	0	Prostopadłe kierunki
120°	–0.5	Kierunki coraz bardziej przeciwne
150°	≈ –0.866	Bardzo przeciwne
180°	–1	Kierunki przeciwnie skierowane

Przykład obliczeniowy

Oblicz cosinus kąta θ dla przedstawionego trójkąta.



Obliczenia

$$\cos\theta = \frac{a}{c} = \frac{4}{5} = \mathbf{0.8}$$

$$\theta = \arccos(0.8) \approx \mathbf{36.87°}$$

Cosinus kąta ≈ 0.8 oznacza, że kierunki są dość mocno zgodne (kąt jest ostry).

Twierdzenie Pitagorasa

W trójkącie prostokątnym suma kwadratów przyprostokątnych jest równa kwadratowi przeciwprostokątnej.

Jak to działa?

1. **A** – wysokość wikinga do barku
2. **B** – odległość od wikinga do marchewki
3. **C** – długość dzidy
4. **A** i **B** to przyprostokątne
5. **C** to przeciwprostokątna

$$A^2 + B^2 = C^2$$

To właśnie opisuje twierdzenie Pitagorasa.



Jeśli znamy **A** i **B**, możemy obliczyć długość dzidy **C**.

Iloczyn skalarny, projekcja i wysokość obiektu

Jak obliczyć, jak wysoko nad graczem znajduje się obiekt na niebie?

1 Mamy wektory



2 Rzutujemy wektor na kierunek patrzenia

Obliczamy rzut wektora **toObject** na forwardVector za pomocą iloczynu skalarnego:

```
projLen = dot(toObject, forwardVector)
```

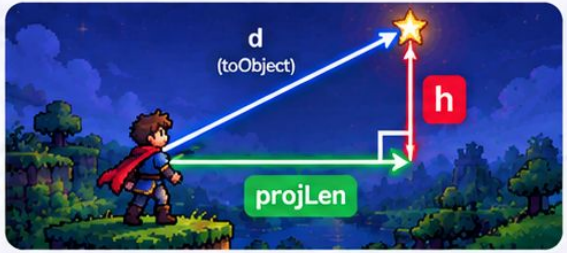


projLen to odległość pozioma (wzdłuż kierunku patrzenia) do punktu pod obiektem na niebie.

3 Liczymy odległość poziomą do obiektu

Długość wektora **toObject** to pełna odległość do obiektu:

```
d = length(toObject)
```



4 Używamy twierdzenia Pitagorasa

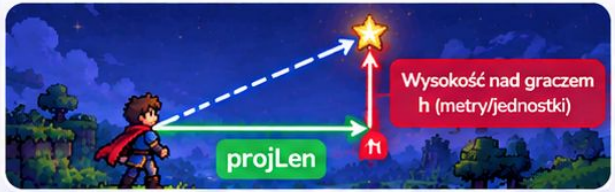
Tworzy się trójkąt prostokątny, gdzie:

- d – przeciwprostokątna (odległość do obiektu)
- projLen – przyprostokątna pozioma
- h – przyprostokątna pionowa (wysokość, której szukamy)

$$h = \sqrt{d^2 - projLen^2}$$

5 Wynik – wysokość obiektu nad graczem

Otrzymana wartość h to wysokość obiektu nad graczem (wzdłuż pionu, prostopadle do kierunku patrzenia).



Pełny wzór

Wysokość h obiektu na niebie:

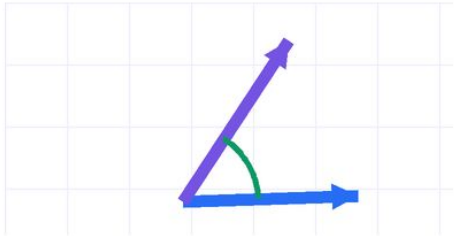
$$h = \sqrt{\underbrace{|toObject|^2}_{\text{Pełna odległość do obiektu}} - \underbrace{\left(dot(toObject, forwardVector)\right)^2}_{\text{Rzut na kierunek patrzenia (odległość pozioma)}}}$$

Dzięki projekcji wektora na kierunek patrzenia i twierdzeniu Pitagorasa możemy szybko obliczyć, jak wysoko nad graczem znajduje się dowolny obiekt na niebie, nawet jeśli nie patrzymy bezpośrednio na niego.

5 Jak interpretować wynik?

Znak wyniku wskazuje wzajemny kierunek niezerowych wektorów.

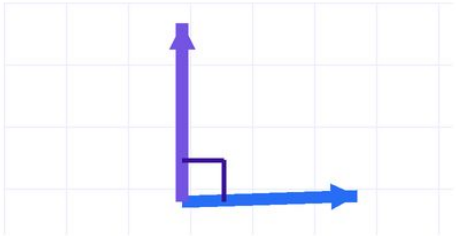
$a \cdot b > 0$



KĄT OSTRY

Podobny kierunek

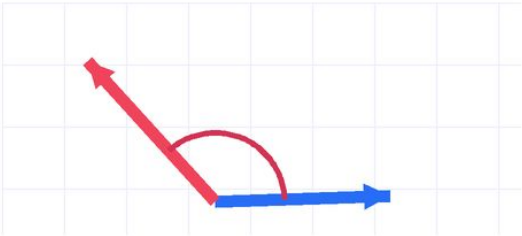
$a \cdot b = 0$



KĄT PROSTY

Wektory prostopadłe

$a \cdot b < 0$



KĄT ROZWARTY

Przeciwny kierunek

Dodatni, zero czy ujemny? To mówi o kącie między wektorami.

Lewa czy prawa strona przeciwnika?

Iloczyn skalarny, iloczyn wektorowy i Blueprint

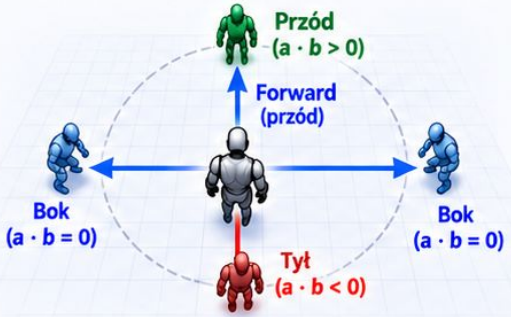
Najważniejsza idea

1. Sam iloczyn skalarny nie mówi, czy przeciwnik jest po lewej czy po prawej.
2. Dot Product określa: **przód / bok / tył.**
3. Aby odróżnić lewo i prawo, użyj **Right Vector** albo znaku **iloczynu wektorowego** (2D: składowa Z).

Dot Product – przód czy tył

$$a \cdot b = |a||b| \cos(\alpha)$$

- $a \cdot b > 0$ → przeciwnik przed graczem
- $a \cdot b = 0$ → przeciwnik dokładnie z boku
- $a \cdot b < 0$ → przeciwnik za graczem

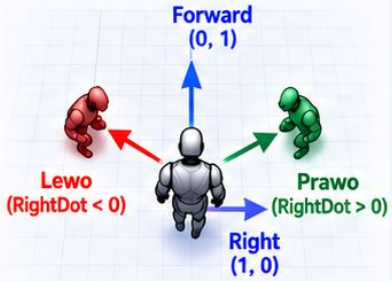


Lewo czy prawo?

Sposób A – Right Vector

`RightDot = RightVector · ToEnemy`

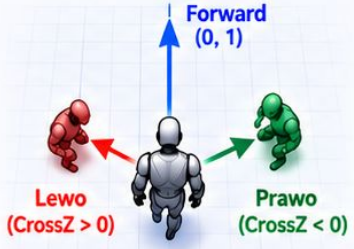
- `RightDot > 0` → przeciwnik po prawej
- `RightDot < 0` → przeciwnik po lewej



Sposób B – Cross Product (2D)

```
CrossZ = Forward.x * ToEnemy.y
       – Forward.y * ToEnemy.x
```

- `CrossZ > 0` → lewa strona
- `CrossZ < 0` → prawa strona



Najprościej w Blueprint: Dot(Forward, ToEnemy) = przód/tył, Dot(Right, ToEnemy) = lewo/prawo.

Przekształcanie wzorów

Wykonujemy te same działania po obu stronach równania.

1 **Start**



2 **To samo działanie po obu stronach**

$$x + 3 \;\overset{-3}{\overset{\downarrow}{=}}\; 7 \;\overset{-3}{\overset{\downarrow}{}}$$

$$x = 7$$

3 **Uproszczony wzór**



Możemy **dodawać**, **odejmować**, **mnożyć** lub **dzielić** obie strony równania.

Przekształcanie wzorów

Przykład z mnożeniem obu stron równania.

1 Start



2 Mnożymy przez 3

×3 ×3

$$\frac{x}{3} = 4$$

3 Wynik



Mnożymy obie strony przez tę samą liczbę.

Przekształcanie wzorów

Przykład z dzieleniem obu stron równania.

1 Start

$$a \cdot b = |a||b| \cos(\alpha)$$



2 Dzielimy przez $|a||b|$

$$a \cdot b = |a||b| \cos(\alpha)$$

$$\frac{a \cdot b}{|a||b|} = \frac{|a||b| \cos(\alpha)}{|a||b|}$$

3 Wynik

$$\cos(\alpha) = \frac{a \cdot b}{|a||b|}$$



Dzielimy obie strony przez ten sam wyraz.